Національний університет «Львівська політехніка»
Кафедра програмного забезпечення

КУРСОВА РОБОТА

з дисципліни «Бази даних»

на тему:

«Інформаційна система для управління IT-проєктами та завданнями»

Виконав:
студент спеціальності 121
«Інженерія програмного забезпечення»
групи П3-37
Кормило В.В.

Керівник:
Старший викладач кафедри програмного забезпечення,
Білоіваненко Максим Вікторович

Оцінка:
Національна шкала _____

Кількість балів _____ Оцінка ECTS ____

Члени комісії _____ Максим БІЛОІВАНЕНКО
(підпис)

_____
(підпис)

Львів – 2026 рік

Зміст

Аналіз предметної області та постановка завдання 3
Опис предметної області 3
Актуальність теми 3
Визначення важливих понять 3
Сфера застосування системи 3
Огляд існуючих аналогічних систем 4
Вимоги до обробки даних 5
Характеристика бізнес-процесів 5
Способи накопичення та аналізу даних 6
Форми введення та виведення даних 7
Постановка завдання 8
Призначення інформаційної системи 8
Ролі потенційних користувачів 8
Перелік функцій 8
Розробка моделей даних 9
Концептуальне проєктування 9
Характеристики понять, що описані в базі даних 9
Концептуальні взаємозв’язки між поняттями 9
Вимоги до системи накопичення даних 10
Оцінка обсягів даних 10
Частота додавання та оновлення 11
Очікувані запити та вибірки 11
Обмеження доступу 11
Логічне проектування бази даних 12
Оптимізація бази даних (Проектування індексів) 17
Реалізація процедур бізнес-логіки 18
Опис інших моделей даних 20
Розробка програмного коду 22
Обґрунтування обраної архітектури 22
Структурна модель інформаційної системи 24
Призначення модулів та компонентів системи 25
Опис реалізації функцій системи 26
Опис реалізації елементів інтерфейсу 27

Аналіз предметної області та постановка завдання

Опис предметної області

Актуальність теми

Актуальність розробки зумовлена потребою команд розробки програмного забезпечення у централізованому інструменті керування проєктами, завданнями та командною взаємодією. У реальних умовах одночасно виконуються десятки задач із різними пріоритетами, виконавцями та строками, що ускладнює контроль прогресу без спеціалізованої інформаційної системи.

Запропонована система дозволяє структурувати роботу за Agile-підходом (спринти, backlog, релізи), забезпечити прозорість виконання, скоротити ризики пропуску дедлайнів та підвищити продуктивність команди.

Визначення важливих понять

- **Користувач** – зареєстрований учасник системи (менеджер, розробник, адміністратор), який виконує операції відповідно до ролі.
- **Проєкт** – логічна одиниця планування робіт із власною командою, спринтами та набором задач.
- **Спринт** – обмежений у часі ітераційний період виконання задач проєкту.
- **Завдання (Issue/Task)** – елемент роботи із назвою, описом, пріоритетом, статусом, виконавцем і терміном виконання.
- **Реліз** – зафіксований результат завершених спринтів або набору задач.
- **Backlog** – перелік задач, що очікують планування або виконання.
- **Scrum-метрики** – показники ефективності (velocity, відсоток виконання, завантаження учасників).

Сфера застосування системи

Система орієнтована на IT-команди, які ведуть розробку програмних продуктів у форматі Agile/Scrum. Вона може використовуватися у:

- навчальних/академічних IT-проєктах;
- малих і середніх командах розробки;

- стартапах для керування roadmap, задачами та релізами;
- внутрішніх відділах розробки компаній.

Огляд існуючих аналогічних систем

- **Jira Software:** вважається галузевим стандартом для технічних команд завдяки глибокій підтримці Scrum/Kanban, гнучким workflow і розвиненій системі ролей, однак її впровадження потребує більше часу та налаштувань, а вартість для команд може бути відчутною.
- **Trello:** забезпечує дуже швидкий старт і простий візуальний підхід у форматі дошок, проте при масштабуванні проєктів виникають обмеження щодо складної аналітики, управління залежностями та детального контролю процесів.
- **Asana:** добре підходить для планування командних задач і координації між відділами, але для суто технічних Scrum-процесів зазвичай поступається Jira у глибині інструментів.

Табл. 1.1.

Порівняльна таблиця існуючих аналогічних систем

Критерій порівняння	**Jira Software**	**Trello**	**Asana**
Основна модель роботи	Scrum/Kanban, enterprise-процеси	Kanban-дошки, прості робочі потоки	Task/project management для кросфункціональних команд
Складність впровадження	Висока (потрібне налаштування)	Низька (швидкий старт)	Середня
Підтримка Scrum	Дуже сильна (спринти, backlog, velocity)	Базова/обмежена	Помірна, менш технічно глибока
Аналітика та звіти	Розширена, гнучка	Обмежена (базова візуалізація)	Добра для менеджменту, слабша для технічного Scrum
Гнучкість workflow	Висока (кастомні стани, правила, автоматика)	Середня (переважно через плагіни)	Середньо-висока

Продовження табл. 1.1.

Вартість для команди	Відносно висока при масштабуванні	Нижча, простіший тарифний поріг	Середня
Придатність для складних IT-проєктів	Висока	Середня/низька	Середня

Вимоги до обробки даних

Характеристика бізнес-процесів

1. **Планування структури проєкту**

   Процес починається зі створення проєкту, визначення команди та ключових параметрів виконання. Далі формується backlog із переліком задач, які деталізуються до рівня, придатного для реалізації. Після пріоритизації задачі розподіляються по спринтах відповідно до цілей і місткості команди. Результатом є структурований план робіт на ітерації.

2. **Розподіл задач**

   На цьому етапі задачам призначаються виконавці з урахуванням компетенцій і поточного навантаження. Для кожної задачі встановлюються пріоритет, дедлайн і відповідальний за постановку. За потреби менеджер перерозподіляє задачі для балансування команди. Це забезпечує прозору відповідальність і керований графік виконання.

3. **Виконання задач**

   Під час реалізації задачі проходять життєвий цикл To Do -> In Progress -> In Review -> Done. Перехід між статусами контролюється бізнес-правилами, щоб уникнути некоректних дій. Усі зміни фіксуються в історії для контролю та аналізу. Такий підхід забезпечує керований і відтворюваний процес розробки.

4. **Контроль прогресу**

   Менеджер і команда відстежують стан спринту через дошку задач та зведені показники. Система виявляє прострочення, ризики зриву дедлайнів і нерівномірне завантаження учасників. За результатами моніторингу

виконуються корекції пріоритетів або перепризначення задач. Це підвищує передбачуваність і стабільність виконання спринту.

5. **Формування релізів**

   Після завершення спринту формується реліз із задач, що мають статус Done і пройшли перевірку. У релізі фіксуються версія, дата та перелік реалізованих змін. Невиконані задачі переносяться в backlog наступної ітерації. Таким чином забезпечується контрольоване постачання функціоналу.

6. **Аналітика**

   Система обчислює ключові метрики: velocity, відсоток виконання спринту та продуктивність виконавців. Звіти дозволяють порівнювати планові й фактичні показники між ітераціями. На основі аналітики команда приймає рішення щодо покращення планування й розподілу навантаження. Додатково підтримується формування консолідованих звітів у PDF.

Способи накопичення та аналізу даних

Накопичення інформації реалізується у реляційній СУБД PostgreSQL, що забезпечує надійне довготривале зберігання даних і підтримку транзакційної цілісності (ACID).

Структура БД поділяється на кілька логічних груп:

- облікові дані (users, roles, user_roles)
- довідкові дані (статуси, пріоритети, типи сутностей)
- транзакційні дані (projects, sprints, tasks, task_history, releases)
- аналітичні результати, що обчислюються на основі операційних таблиць.

Аналіз даних виконується за допомогою агрегатних SQL-запитів і представлень:

- обчислюється sprint velocity (кількість/вага завершених задач за спринт);
- відсоток виконання спринту;

- структура завантаження по виконавцях і динаміка переходів задач між статусами.

Форми введення та виведення даних

Введення даних здійснюється через графічні форми користувацького інтерфейсу:

- реєстрація та авторизація;
- створення/редагування проєктів;
- планування спринтів;
- постановка задач;
- призначення виконавців;
- зміна статусів і формування релізів.

Для ключових операцій передбачена валідація (обов’язкові поля, коректність дат, допустимі переходи статусів, права доступу за ролями), а у разі помилок користувач отримує зрозумілі повідомлення.

Виведення даних реалізоване у форматі, зручному для щоденної роботи команди:

- kanban-дошка для оперативного контролю;
- backlog для планування;
- Сторінка спринту для ітераційного моніторингу;
- Табличний список задач із фільтрами за проєктом, виконавцем, пріоритетом і статусом.
- Для керівних ролей доступна панель аналітики з ключовими метриками та трендами, а також функція формування зведених звітів з експортом у PDF.

Постановка завдання

Призначення інформаційної системи

Призначення системи – забезпечити єдине цифрове середовище для управління IT-проєктами та завданнями, у якому керівники і команда можуть планувати спринти, контролювати виконання робіт, формувати релізи та аналізувати ефективність процесу розробки.

Ролі потенційних користувачів

- Адміністратор: керує користувачами, ролями, довідниками, системними налаштуваннями та правами доступу.
- Менеджер проєкту: створює проєкти, спринти, задачі, призначає виконавців, контролює терміни, формує релізи.
- Розробник: переглядає призначені задачі, змінює статуси, додає коментарі, звітує про виконання.

Перелік функцій

Основні функції управління даними, які доступні через інтерфейс користувача:

1. Створення, редагування та архівування проєктів.
2. Створення і керування спринтами.
3. Створення і керування задачами.
4. Пошук і фільтрація задач за проєктом, спринтом, статусом, пріоритетом, виконавцем.
5. Створення релізів на основі завершених задач/спринтів.
6. Формування аналітичних звітів.

Додаткова функція:

1. Експорт консолідованих звітів про продуктивність команди у PDF.

Розробка моделей даних

Концептуальне проєктування

Характеристики понять, що описані в базі даних

1. **Облікові дані та доступ.**

   У системі зберігаються користувачі, ролі та зв'язок між ними (user-role). Така модель дозволяє реалізувати рольове розмежування доступу (Admin, Project Manager, Developer) і контролювати дозволені дії. Облікові дані включають базову інформацію користувача та захищене зберігання паролів (хешування).

2. **Організація роботи.**

   Організаційний блок охоплює команди, учасників команд, проєкти та спринти. Проєкт прив'язується до команди, а спринти визначають ітераційні межі виконання задач. Це забезпечує структуроване планування та прозорий розподіл відповідальності.

3. **Операційні сутності.**

   Основною робочою сутністю є завдання з атрибутами пріоритету, статусу, виконавця та дедлайну. Історія змін статусів фіксується окремо для аудиту та відстеження прогресу. Релізи формуються на основі завершених задач/спринтів і відображають підсумок ітерації.

4. **Аналітика.**

   Аналітичні показники обчислюються агрегатними запитами на основі операційних таблиць. Система розраховує відсоток виконання спринту, velocity та завантаження учасників. Це дає змогу оцінювати ефективність команди та коригувати планування наступних ітерацій.

Концептуальні взаємозв'язки між поняттями

1. Один проєкт має багато спринтів і багато задач.
2. Один спринт належить одному проєкту та містить багато задач.
3. Одна задача належить одному проєкту та (опційно) одному спринту.

4. Користувач може бути виконавцем багатьох задач.
5. Один реліз пов’язаний з одним проєктом і, за потреби, конкретним спринтом.
6. Історія змін містить багато записів для однієї задачі (1:N).

Рис. 2.1. Логічна схема реляційної бази даних (ER-діаграма)

Вимоги до системи накопичення даних

Оцінка обсягів даних

- **Довідкові та облікові дані (roles, users, teams)** матимуть відносно невеликий або помірний обсяг: таблиця users – від сотень до тисяч записів, roles – фіксований малий довідник, teams – десятки/сотні записів.
- **Організаційні дані (projects, sprints)** зростатимуть стабільно: projects – десятки або сотні записів, sprints – сотні або тисячі, оскільки для кожного проєкту створюється багато ітерацій.
- **Основний транзакційний масив (tasks)** буде найбільш об’ємним серед операційних таблиць: від кількох тисяч до десятків тисяч записів залежно від тривалості життєвого циклу системи.

- **Історичні дані (task_history)** формуватимуться найінтенсивніше: кожна зміна статусу створює новий запис, тому обсяг цієї таблиці може перевищувати tasks у 2–5 разів.

Частота додавання та оновлення

- **Низька частота:** довідники, ролі, базові параметри (змінюються рідко, переважно адміністратором).
- **Середня частота:** створення нових проєктів, спринтів та оновлення їхніх атрибутів у процесі планування.
- **Висока частота:** операції із задачами (створення, редагування, зміна виконавця), переходи між статусами, фіксація історії та службових коментарів.

Найбільш динамічним сегментом є саме життєвий цикл задач, тому система повинна бути оптимізована для частих записів і швидкого читання актуального стану.

Очікувані запити та вибірки

- **Оперативні вибірки:** виведення задач конкретного проєкту або спринту для kanban/backlog-представлення.
- **Фільтрація:** відбір задач за статусом, пріоритетом, виконавцем, дедлайном та комбінацією цих параметрів.
- **Контроль термінів:** пошук прострочених задач (дедлайн минув, статус не Done) і задач із високим ризиком невиконання.
- **Аналітичні запити:** формування звітів «виконано/невиконано», обчислення velocity, завантаження учасників, динаміки переходів між статусами.

Обмеження доступу

- **Адміністратор (Admin):** повний доступ до всіх сутностей, керування ролями, користувачами та системними довідниками.

- **Менеджер проєкту (Project Manager):** створення й редагування проєктів, спринтів, задач та релізів у межах призначених проєктів; контроль виконання команди.
- **Розробник (Developer):** перегляд доступних проєктів і зміна лише тих задач, які призначені йому або дозволені правилами проєкту.

Неавторизовані користувачі не мають прав на модифікацію даних; усі операції виконуються лише після автентифікації, а дозвіл на дію перевіряється на сервері відповідно до ролі та контексту проєкту.

Логічне проектування бази даних

Табл. 2.1.

Структура таблиці users (Користувачі)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор користувача
full_name	VARCHAR(150)	NOT NULL	Повне ім’я користувача
email	VARCHAR(255)	UNIQUE, NOT NULL	Електронна пошта (логін)
password_hash	VARCHAR(255)	NOT NULL	Хеш пароля користувача
position	VARCHAR(100)	NOT NULL	Посада користувача
is_active	BOOLEAN	NOT NULL, DEFAULT TRUE	Ознака активності акаунта
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата та час створення запису

Табл. 2.2.

Структура таблиці roles (Ролі)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	SMALLSERIAL	PK	Унікальний ідентифікатор ролі

Продовження табл. 2.2.

name	VARCHAR(50)	UNIQUE, NOT NULL	Назва ролі (Admin, Project Manager, Developer)
description	VARCHAR(255)	NULL	Опис призначення ролі

Табл. 2.3.

Структура таблиці user_roles (Зв’язок користувачів і ролей)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
user_id	BIGINT	PK, FK -> users(id), NOT NULL	Посилання на користувача
role_id	SMALLINT	PK, FK -> roles(id), NOT NULL	Посилання на роль

Примітка: первинний ключ є складеним: user_id + role_id.

Табл. 2.4.

Структура таблиці teams (Команди)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор команди
name	VARCHAR(120)	UNIQUE, NOT NULL	Назва команди
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата створення команди

Табл. 2.5.

Структура таблиці team_members (Учасники команди)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
team_id	BIGINT	PK, FK -> teams(id), NOT NULL	Посилання на команду
user_id	BIGINT	PK, FK -> users(id), NOT NULL	Посилання на користувача
joined_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата входу користувача в команду

Примітка: первинний ключ є складеним: team_id + user_id.

Табл. 2.6.

Структура таблиці projects (Проєкти)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор проєкту
team_id	BIGINT	FK -> teams(id), NOT NULL	Команда, що виконує проєкт
name	VARCHAR(180)	NOT NULL, UNIQUE(team_id, name)	Назва проєкту
description	TEXT	NULL	Опис проєкту
start_date	DATE	NULL	Планова дата початку
end_date	DATE	NULL, CHECK end_date >= start_date	Планова дата завершення
status	VARCHAR(20)	NOT NULL, CHECK	Статус (planned, active, on_hold, completed, archived)
created_by	BIGINT	FK -> users(id), NOT NULL	Користувач, який створив проєкт

Табл. 2.7.

Структура таблиці sprints (Спринти)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор спринту
project_id	BIGINT	FK -> projects(id), NOT NULL	Посилання на проєкт
name	VARCHAR(120)	NOT NULL, UNIQUE(project_id, name)	Назва спринту

Продовження табл. 2.7.

start_date	DATE	NOT NULL	Дата початку спринту
end_date	DATE	NOT NULL, CHECK end_date >= start_date	Дата завершення спринту
goal	TEXT	NULL	Ціль спринту
status	VARCHAR(20)	NOT NULL, CHECK	Статус (planned, active, closed, cancell ed)

Табл. 2.8.

Структура таблиці tasks (Завдання)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор задачі
project_id	BIGINT	FK -> projects(id), NOT NULL	Проєкт, до якого належить задача
sprint_id	BIGINT	FK -> sprints(id), NULL	Спринт задачі (може бути NULL для backlog)
title	VARCHAR(220)	NOT NULL	Коротка назва задачі
description	TEXT	NULL	Детальний опис задачі
priority	VARCHAR(20)	NOT NULL, CHECK	Пріоритет (low, medium, high, critical)
status	VARCHAR(20)	NOT NULL, CHECK	Статус (todo, in_progress, in_review, done, blocked)
story_point	int	CHECK	Оцінка складності завдання
assignee_id	BIGINT	FK -> users(id), NULL	Виконавець задачі
reporter_id	BIGINT	FK -> users(id), NOT NULL	Постановник задачі
due_date	DATE	NULL	Кінцевий термін виконання

Табл. 2.9.

Структура таблиці task_comments (Коментарі до завдань)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор коментаря
task_id	BIGINT	FK -> tasks(id), NOT NULL	Завдання, до якого належить коментар
author_id	BIGINT	FK -> users(id), NOT NULL	Користувач, який додав коментар
body	TEXT	NOT NULL	Контент коментаря
created_at	TIMESTAMPTZ	NOT NULL	Дата створення коментаря
updated_at	TIMESTAMPTZ	NOT NULL	Дата оновлення коментаря

Табл. 2.10.

Структура таблиці releases (Релізи)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор релізу
project_id	BIGINT	FK -> projects(id), NOT NULL	Проєкт, у межах якого сформовано реліз
sprint_id	BIGINT	FK -> sprints(id), UNIQUE, NULL	Спринт-джерело релізу
version	VARCHAR(30)	NOT NULL, UNIQUE(project_id, version)	Версія релізу (напр. v1.2.0)
release_date	DATE	NOT NULL	Дата публікації релізу
notes	TEXT	NULL	Примітки до релізу (release notes)

Табл. 2.11.

Структура таблиці task_history (Історія змін статусів задач)

Назва поля	**Тип даних (домен)**	**Ключі / обмеження**	**Опис атрибута**
id	BIGSERIAL	PK	Унікальний ідентифікатор запису історії
task_id	BIGINT	FK -> tasks(id), NOT NULL	Задача, для якої зафіксовано зміну
changed_by	BIGINT	FK -> users(id), NOT NULL	Користувач, що виконав зміну
old_status	VARCHAR(20)	NOT NULL, CHECK	Попередній статус задачі
new_status	VARCHAR(20)	NOT NULL, CHECK	Новий статус задачі
changed_at	TIMESTAMPTZ	NOT NULL, DEFAULT now()	Дата та час зміни статусу

Оптимізація бази даних (Проектування індексів)

1. Композитний індекс для дошки проєкту:

- Таблиця: tasks
- Поля: project_id, status
- Обґрунтування з точки зору UI: Найчастіший запит у системі – відкриття дошки задач конкретного проєкту, де задачі групуються за статусами (To Do, In Progress, In Review, Done). Індекс прискорює відбір записів у межах проєкту та їх сортування/групування за станом, зменшуючи час завантаження основного робочого екрана.

2. Композитний індекс для особистого кабінету виконавця:

- Таблиця: tasks
- Поля: assignee_id, status
- Обґрунтування з точки зору UI: У розділі «Мої задачі» користувач постійно переглядає лише призначені йому задачі з поділом за статусом. Індекс дозволяє швидко виконувати вибірки виду «усі активні задачі конкретного виконавця» і підтримує оперативне перемикання фільтрів у інтерфейсі.

3. Індекс для сторінки спринту та backlog:

- Таблиця: tasks
- Поля: sprint_id
- Обґрунтування з точки зору UI: При відкритті сторінки спринту система формує перелік задач за конкретною ітерацією. Окремий індекс за sprint_id оптимізує запити, пов'язані з наповненням backlog/board спринту, особливо коли загальний обсяг задач у проєкті зростає до десятків тисяч.

4. Композитний індекс для календаря спринтів:

- Таблиця: sprints
- Поля: project_id, start_date, end_date
- Обґрунтування з точки зору UI: Екрани планування використовують вибірки спринтів у межах одного проєкту з урахуванням часових меж. Індекс прискорює запити для календарного відображення ітерацій, перевірки перетинів дат та формування хронології спринтів.

5. Композитний індекс для журналу змін задачі:

- Таблиця: task_history
- Поля: task_id, changed_at
- Обґрунтування з точки зору UI: На сторінці детального перегляду задачі часто відображається хронологія зміни статусів і дій користувачів. Індекс забезпечує швидке отримання історії конкретної задачі у часовому порядку, що критично для аудиту та аналізу затримок у workflow.

Реалізація процедур бізнес-логіки

1. Представлення vw_project_task_summary

- Призначення:

  Оптимізація аналітичних запитів для дашборду менеджера проєкту. Представлення повертає зведену статистику по проєкту: загальну кількість задач, кількість завершених, частку виконання та кількість прострочених.
- Логіка роботи:

Представлення агрегує таблицю tasks у розрізі project_id, використовуючи умовні підрахунки (CASE WHEN) за статусами та дедлайнами. Дані формуються в одному запиті, що зменшує навантаження на прикладний рівень і прискорює побудову панелі аналітики у UI.

2. Представлення vw_sprint_velocity

- Призначення:

  Надання готового джерела даних для розрахунку velocity по спринтах без дублювання бізнес-формул у серверному коді.

- Логіка роботи:

  Для кожного спринту обчислюється кількість задач зі статусом done та відсоток виконання (done / all * 100). Представлення об'єднує sprints і tasks, після чого групує результати за sprint_id. Це забезпечує стандартизований і відтворюваний підхід до метрик спринту.

3. Збережена процедура sp_close_sprint(p_sprint_id BIGINT)

- Призначення:

  Транзакційне закриття спринту з контролем цілісності даних і коректним завершенням ітерації.

- Логіка роботи:

  Процедура перевіряє існування спринту та його поточний статус; закрити можна лише активний спринт. Далі система фіксує підсумкові показники, змінює статус спринту на closed і переводить незавершені задачі до backlog (наприклад, встановлення sprint_id = NULL) або в наступний спринт за правилами системи. Усі операції виконуються в межах однієї транзакції; у разі помилки зміни скасовуються (ROLLBACK).

4. Збережена процедура sp_create_release_from_sprint(p_sprint_id BIGINT, p_version VARCHAR)

- Призначення:

  Автоматизоване створення релізу на основі завершеного спринту з перевіркою бізнес-умов.

- Логіка роботи:

Процедура отримує ідентифікатор спринту та номер версії релізу, перевіряє, що спринт має статус closed, а версія ще не існує в межах проєкту. Після валідації створюється запис у releases із датою формування та службовими нотатками. Це унеможливлює дублювання релізів і гарантує, що реліз формується лише з коректно завершеної ітерації.

5. Тригер trg_task_status_history + функція fn_log_task_status_change()

- Призначення:

  Автоматичне ведення журналу змін статусів задач для аудиту, аналітики та контролю процесу виконання.

- Логіка роботи:

  Тригер спрацьовує при UPDATE поля status у таблиці tasks. Якщо значення статусу реально змінилося, у task_history додається новий запис із попереднім статусом, новим статусом, користувачем ініціатором (передається прикладним рівнем) та часом зміни. Це забезпечує повну трасованість життєвого циклу задачі та уніфікує фіксацію подій на рівні БД.

Опис інших моделей даних

1. Модель обміну даними (REST API + JSON)

Взаємодія між клієнтським інтерфейсом і серверною частиною реалізується за архітектурним стилем REST через HTTP(S). Основним форматом передачі даних є JSON, що дозволяє уніфіковано передавати структури проєктів, спринтів, задач, релізів і аналітичних показників.

Для отримання даних застосовуються GET-запити, для створення – POST, для оновлення – PUT/PATCH, для видалення – DELETE. Такий підхід забезпечує чіткий контракт між фронтендом і бекендом, а також спрощує тестування endpoint-ів.

2. Прикладна модель доступу до БД (Node.js + Express + Prisma ORM)

Серверна логіка реалізується на Node.js із фреймворком Express, а доступ до PostgreSQL виконується через Prisma ORM. Prisma надає типобезпечний шар

доступу до даних, де таблиці БД відображаються у вигляді моделей (User, Project, Sprint, Task, Release тощо).

Це дозволяє централізувати CRUD-операції, валідацію зв'язків та транзакційні сценарії (наприклад, закриття спринту й перенесення незавершених задач). Додатково Prisma Migrate використовується для керованої еволюції схеми БД через версіоновані міграції.

3. Модель передачі даних у застосунку (DTO)

Для розділення внутрішньої структури БД та зовнішнього API використовуються DTO-об'єкти (Data Transfer Objects). Наприклад, у відповідях API не повертаються службові поля на кшталт password_hash, натомість формуються безпечні та зручні для UI структури (taskCardDto, sprintSummaryDto). Це дає змогу стабілізувати API-контракт навіть при зміні внутрішньої схеми таблиць. Також DTO спрощують валідацію вхідних даних і зменшують ризик неузгодженості між фронтендом і бекендом.

4. Модель автентифікації та доступу (JWT + RBAC)

Для керування сесіями та правами доступу використовується JWT (JSON Web Token). Після успішного входу сервер формує токен, що містить ідентифікатор користувача та роль (Admin, Project Manager, Developer), а клієнт передає його в заголовку Authorization. На рівні middleware Express виконується перевірка токена, після чого контролери застосовують рольову модель доступу (RBAC) до конкретних endpoint-ів. Така схема забезпечує контроль доступу до операцій зміни даних і відповідає вимогам безпеки інформаційної системи.

5. Модель аналітичних даних і звітності

Аналітичні показники не дублюються окремим сховищем, а обчислюються на основі операційних таблиць PostgreSQL через агрегатні SQL-запити, представлення або Prisma raw queries.

До ключових метрик належать velocity, частка завершених задач у спринті, кількість прострочених задач і завантаження виконавців. Результати агрегуються на сервері та передаються клієнту у вигляді структур JSON для відображення на

дашбордах. За потреби ці самі дані використовуються для формування консолідованих звітів з експортом у PDF.

Розробка програмного коду

Обґрунтування обраної архітектури

Для реалізації системи обрано трирівневу клієнт-серверну архітектуру: клієнт (React) – сервер (Node.js + Express.js) – база даних (PostgreSQL). Такий підхід дозволяє винести бізнес-логіку на сервер, забезпечити централізований контроль доступу та спростити масштабування системи. Розмежування шарів зменшує зв'язність компонентів і полегшує супровід коду. Застосування Prisma ORM додатково підвищує надійність роботи з БД завдяки типобезпечному доступу до даних.

1. Мови програмування (JavaScript / TypeScript)

Для розробки клієнтської та серверної частин використано JavaScript і TypeScript.

- Обґрунтування: JavaScript забезпечує єдину технологічну основу для full-stack розробки, а TypeScript додає статичну типізацію, що зменшує кількість runtime-помилок і покращує підтримуваність коду. Використання типів особливо важливе для API-контрактів, DTO та складних об'єктів предметної області.

2. Технології клієнтського інтерфейсу (React)

Клієнтська частина реалізується як односторінковий додаток (SPA) на React.

- Обґрунтування: React підтримує компонентний підхід, що спрощує повторне використання UI-елементів (форми задач, списки, модальні вікна, дошки спринтів). Завдяки маршрутизації та керуванню станом забезпечується динамічна робота інтерфейсу без повного перезавантаження сторінки.

3. Серверна платформа (Node.js + Express.js)

Серверна частина будується на Node.js із використанням фреймворку Express.js.

- Обґрунтування: Express надає гнучкий механізм побудови REST API, маршрутизації та middleware для автентифікації, валідації і централізованої обробки помилок. Це дозволяє ефективно реалізувати бізнес-логіку системи керування задачами, спринтами й релізами.

4. Система управління базами даних (PostgreSQL)

Як основне сховище даних використовується реляційна СУБД PostgreSQL.

- Обґрунтування: PostgreSQL підтримує транзакції ACID, зовнішні ключі, CHECK-обмеження, індексацію та складні агрегатні запити, що є критичним для цілісності операційних даних і побудови аналітики. СУБД добре підходить для систем із активними CRUD-операціями та історичними записами.

5. Шар доступу до даних (Prisma ORM)

Для взаємодії серверної частини з PostgreSQL використовується Prisma ORM.

- Обґрунтування: Prisma забезпечує типобезпечні запити до БД, централізоване описання моделей і керування міграціями схеми. Це підвищує швидкість розробки, зменшує ризик SQL-помилок і забезпечує узгодженість між кодом застосунку та структурою бази даних.

6. Протокол та формат обміну даними (REST + JSON)

Взаємодія між клієнтом і сервером відбувається через REST API з передачею даних у форматі JSON.

- Обґрунтування: Використання стандартних HTTP-методів (GET, POST, PUT/PATCH, DELETE) робить API передбачуваним і зручним для тестування. JSON забезпечує просту серіалізацію/десеріалізацію даних та сумісність із React, Node.js і Prisma-структурами.

Структурна модель інформаційної системи

Інформаційна система реалізується як багаторівневий застосунок, у якому клієнтська частина, серверна логіка та база даних працюють як єдина узгоджена екосистема. Користувач взаємодіє з графічним інтерфейсом, виконує операції з проєктами, спринтами і задачами, а всі зміни зберігаються у реляційній БД PostgreSQL через API. Така архітектура забезпечує розділення відповідальності між шарами, підвищує надійність та спрощує розвиток функціоналу. Основна перевага підходу — централізоване виконання бізнес-правил і стабільна робота інтерфейсу при активній роботі з даними.

- Клієнтський рівень (Frontend, React): складається зі сторінок, компонентів інтерфейсу, маршрутизації та керування станом.

  Виконує введення, відображення й оновлення даних без перезавантаження сторінки та використовується для щоденної роботи користувачів із проєктами, спринтами і задачами.
- Прикладний рівень (API, Node.js + Express): складається з маршрутів, контролерів, сервісів, middleware і шару доступу до БД.

  Виконує бізнес-логіку, валідацію, авторизацію та обробку запитів і використовується для централізованого керування правилами системи.
- Рівень даних (PostgreSQL + Prisma): складається з реляційної схеми таблиць, зв’язків, обмежень цілісності, індексів, процедур і тригерів.

  Виконує надійне зберігання та узгоджену обробку даних і використовується для підтримки транзакцій, історії змін та аналітичних вибірок.
- Модель взаємодії між рівнями: складається з послідовності «інтерфейс -> API -> база даних -> API -> інтерфейс».

  Виконує узгоджений обмін даними між компонентами системи і використовується для забезпечення швидкої та передбачуваної реакції на дії користувача.
- Модульна функціональна структура системи: складається з модулів автентифікації, проєктів/спринтів, задач, релізів та аналітики.

Виконує розподіл системи на логічно незалежні частини і використовується для зручного супроводу, масштабування та поетапного розвитку функціоналу.

Призначення модулів та компонентів системи

1. Модуль автентифікації та авторизації

- Призначення: Забезпечення безпечного доступу до системи та розмежування прав користувачів відповідно до ролей.
- Функціонал: Модуль реалізує реєстрацію та вхід користувачів, перевірку облікових даних, формування сесії доступу та контроль ролей (Admin, Project Manager, Developer). На рівні API застосовуються middleware-компоненти, що перевіряють токен автентифікації та обмежують доступ до endpoint-ів згідно з роллю. Також модуль відповідає за захист приватних маршрутів і повернення стандартизованих помилок доступу (401/403) у разі порушення правил безпеки.

2. Модуль управління проєктами і спринтами

- Призначення: Організація структури робіт у межах проєкту та підтримка ітераційного планування.
- Функціонал: Модуль надає можливість створювати, редагувати, архівувати проєкти та формувати спринти з ціллю, датами і статусом. У межах бізнес-валідації перевіряється коректність часових меж (дата завершення не раніше дати початку), допустимість переходів статусів спринту та належність спринту до конкретного проєкту. Додатково модуль забезпечує логіку закриття спринту й підготовки даних для наступної ітерації.

3. Модуль задач

- Призначення: Реалізація повного життєвого циклу задач від постановки до завершення.
- Функціонал: Модуль виконує CRUD-операції для задач, призначення виконавців, зміну пріоритету та статусу, а також підтримує роботу з дедлайнами. Перед зміною стану застосовується бізнес-перевірка

допустимих переходів (To Do -> In Progress -> In Review -> Done) і фіксація історії змін для аудиту. Для користувацького інтерфейсу надаються механізми фільтрації, сортування та пошуку задач за проєктом, спринтом, виконавцем, пріоритетом і статусом.

4. Модуль релізів

- Призначення: Формалізація результатів завершених ітерацій та керування версіями продукту.
- Функціонал: Модуль створює реліз на основі завершеного спринту, фіксує номер версії, дату випуску та текст приміток (release notes). Перед створенням перевіряється, що спринт має допустимий фінальний статус, а номер версії не дублюється в межах проєкту. Це дозволяє підтримувати послідовну історію змін і забезпечує прозорий зв'язок між виконаними задачами та конкретним релізом.

5. Модуль аналітики

- Призначення: Оцінка ефективності виконання робіт та підтримка управлінських рішень на основі даних.
- Функціонал: Модуль формує агреговані показники, зокрема velocity, відсоток виконання спринту, кількість прострочених задач і завантаження виконавців. Результати обчислюються на основі операційних таблиць і відображаються у вигляді дашбордів, табличних зведень та звітів. Отримані метрики використовуються для ретроспективи, коригування планування наступних спринтів і підвищення передбачуваності виконання проєкту.

Опис реалізації функцій системи

Життєвий цикл виконання ключових функцій системи можна розглянути на прикладі операції зміни статусу задачі. Цей сценарій охоплює взаємодію клієнтської частини, серверної бізнес-логіки та бази даних, тому є показовим для всієї архітектури застосунку.

Користувач у React-інтерфейсі (на kanban-дошці або на сторінці задачі) ініціює зміну статусу, наприклад In Progress -> In Review. Після цієї дії

клієнтський застосунок формує HTTP-запит (PATCH або PUT) до відповідного endpoint-а Express API, передаючи ідентифікатор задачі, новий статус і токен доступу користувача. На рівні інтерфейсу виконується базова валідація, а у разі помилки користувач одразу отримує повідомлення без перезавантаження сторінки.

Після отримання запиту серверний рівень виконує послідовну перевірку: автентичність токена, роль користувача та його право на зміну конкретної задачі. Далі сервісний шар перевіряє бізнес-умови переходу статусу (наприклад, заборону нелогічних переходів або завершення заблокованої задачі без розблокування). Якщо всі умови виконано, оновлення виконується через Prisma у PostgreSQL; для критичних сценаріїв використовується транзакція, щоб гарантувати атомарність змін.

Паралельно або одразу після оновлення основного запису системою фіксується історія зміни статусу (старий стан, новий стан, час і користувач, який виконав дію). Це забезпечує трасованість життєвого циклу задачі, підтримує аудит і дозволяє надалі формувати аналітичні вибірки за переходами статусів. У разі порушення обмежень сервер повертає уніфіковану помилку з відповідним HTTP-кодом і поясненням для користувача.

Після успішного завершення операції API повертає оновлену модель задачі у форматі JSON. React-клієнт оновлює локальний стан і миттєво відображає задачу в новій колонці kanban-дошки, не виконуючи повного перезавантаження сторінки. Завдяки такій реалізації забезпечується швидка реакція інтерфейсу, цілісність даних і передбачувана поведінка системи в щоденній роботі команди.

Опис реалізації елементів інтерфейсу

Інтерфейс системи реалізовано у форматі односторінкового веб-застосунку з лівою панеллю навігації, верхнім службовим рядком і центральною робочою областю контенту. Візуальна концепція побудована у мінімалістичному стилі: світлий фон, нейтральна типографіка, сині акценти для основних дій, компактні форми та помірні заокруглення елементів. Такий підхід забезпечує швидке

сприйняття інформації та зручну щоденну роботу з великою кількістю задач і проєктів.

1. Глобальна структура інтерфейсу (Layout + Navigation)

Інтерфейс складається з фіксованого лівого сайдбару, де розміщено основні розділи системи: «Проєкти», «Спринти», «Дошка задач», «Мої задачі», «Релізи», «Аналітика», «Адміністрування». Активний пункт меню візуально виділяється фоном та акцентним кольором, що покращує орієнтацію користувача в системі. У нижній частині сайдбару відображається блок поточного користувача (ім'я, роль, аватар-ініціали). Верхня панель містить контекстну службову інформацію (дата, заголовок сторінки), а центральна область адаптується під функціонал конкретного модуля.

2. Авторизація та реєстрація

Обидві сторінки оформлені як компактний центрований блок із заголовком і коротким поясненням: на авторизації – поля email і пароль, кнопка входу, посилання на реєстрацію та повідомлення про помилки входу; на реєстрації – поля ПІБ, email, пароль і підтвердження пароля, вибір ролі, кнопка створення акаунта, посилання на вхід і зрозумілі підказки валідації (формат пошти, сила пароля, збіг паролів, зайнятий email).

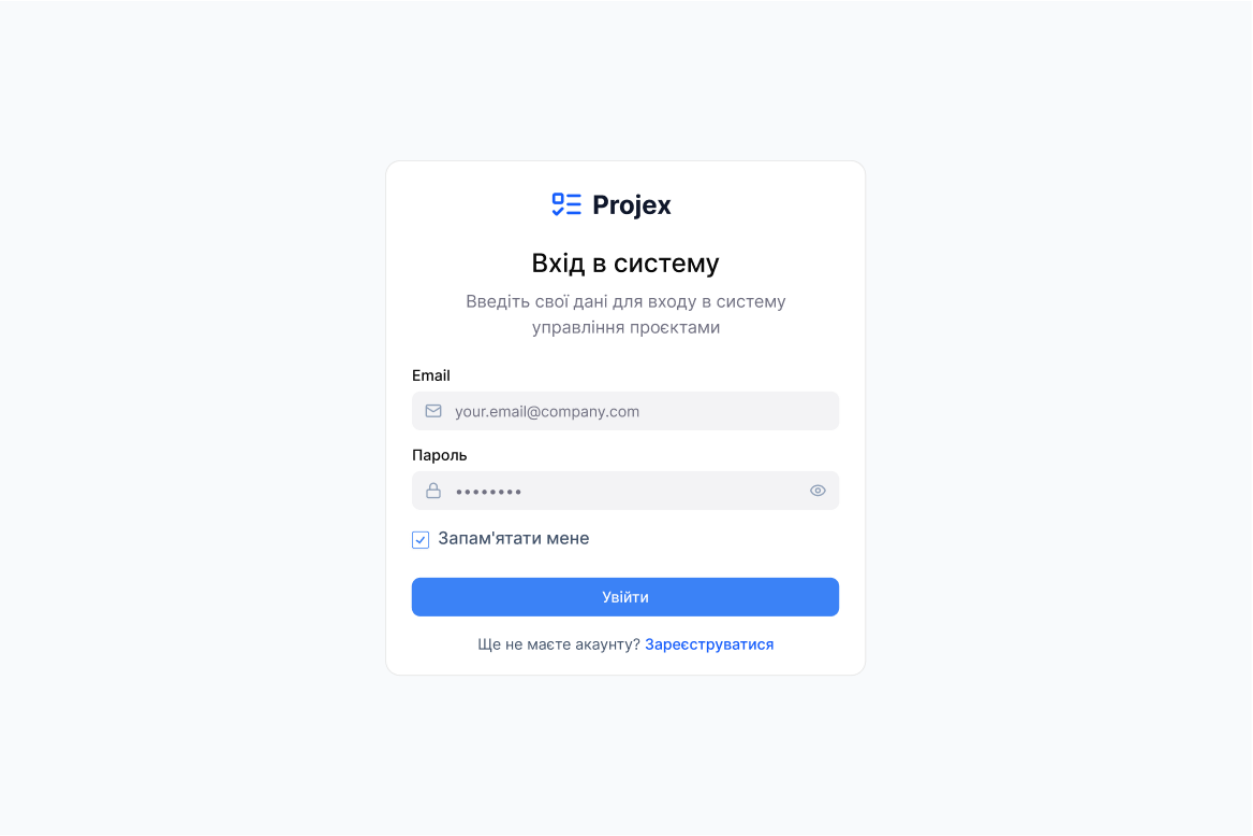

Рис. 3.1. Сторінка входу в систему

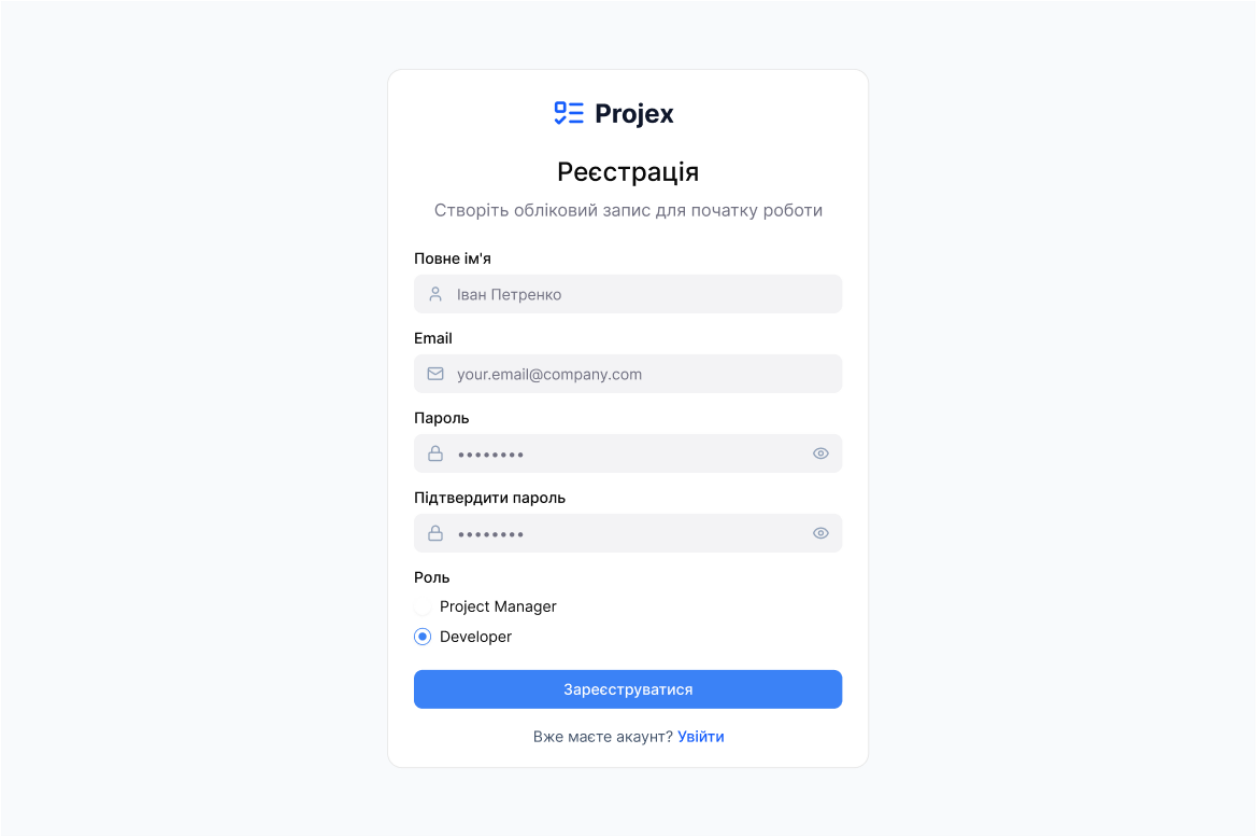

Рис. 3.2. Сторінка реєстрації

3. Сторінка «Проєкти» (керування проєктами команди)

Основний контент сторінки складається із заголовка, підзаголовка, панелі швидких дій та сітки карток проєктів. У верхній частині розміщено кнопку створення нового проєкту, поле пошуку та фільтр за статусом, що дозволяє швидко звузити вибірку. Кожна картка проєкту містить назву, статус, короткий опис, агреговані метрики (кількість задач, учасників, активних спринтів), відповідального менеджера та дату оновлення/створення.

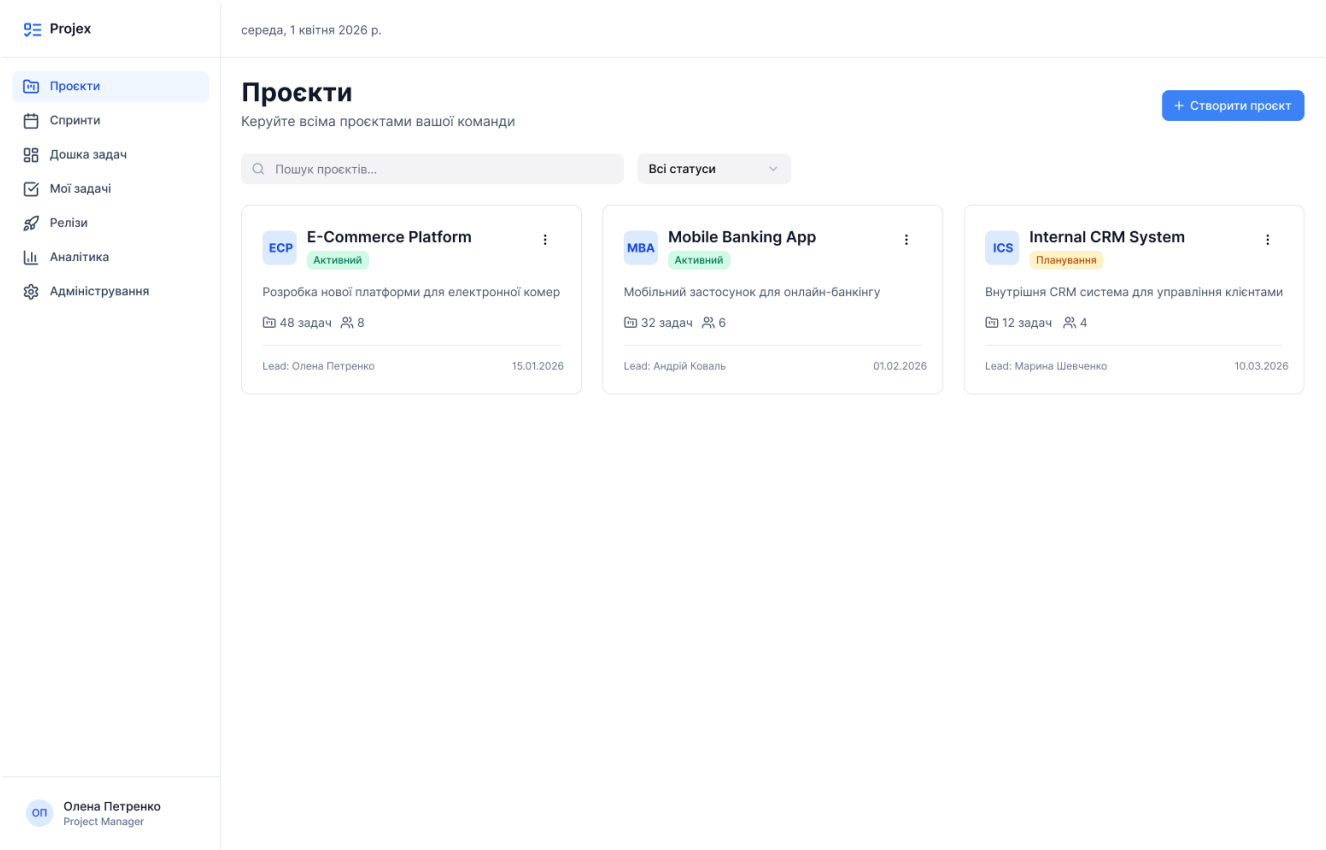

Рис. 3.3. Сторінка проєктів

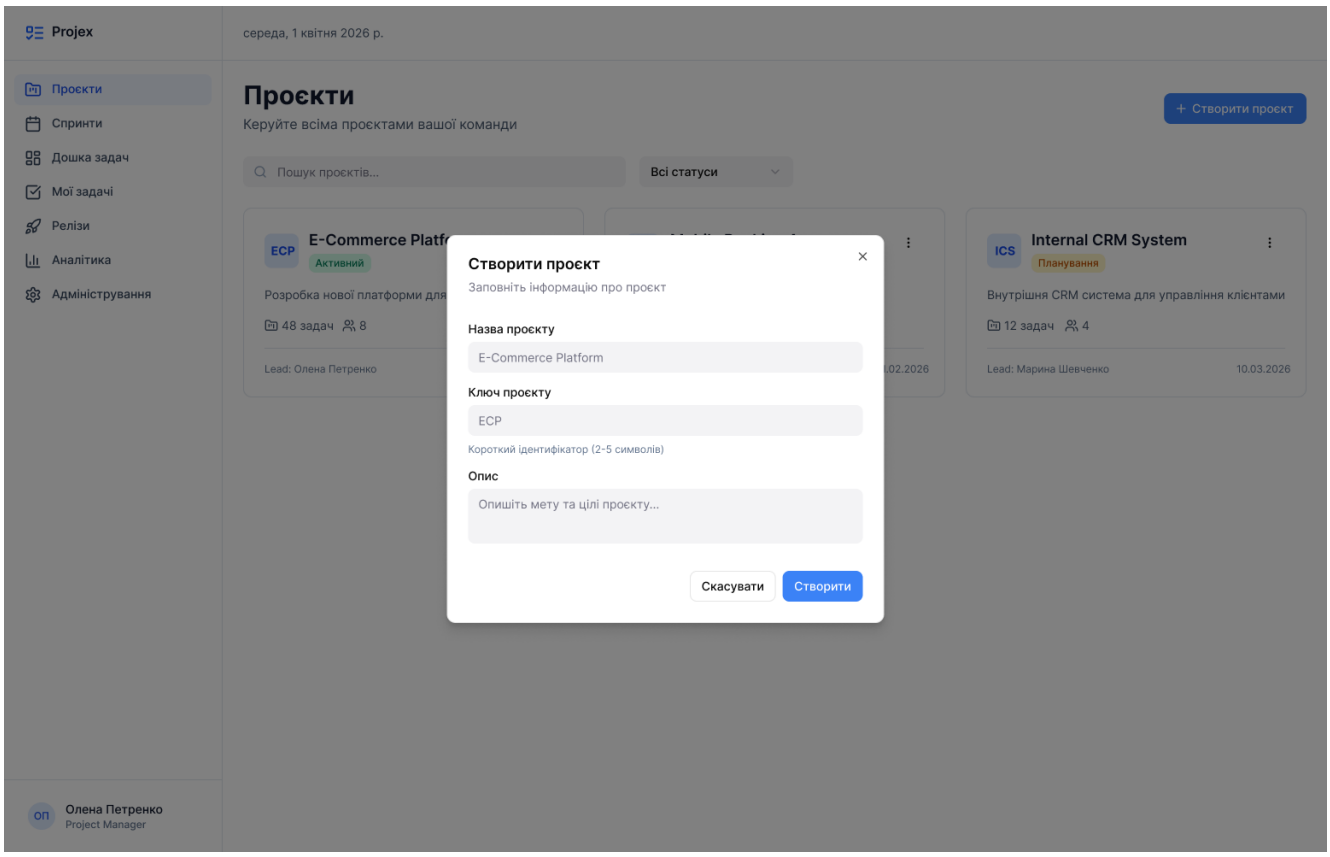

Рис. 3.4. Модальне вікно створення проєкту

4. Сторінка «Спринти» (ітераційне планування)

Інтерфейс спринтів реалізований як робоча зона планування з переліком спринтів і їх ключовими атрибутами: назва, період, ціль, статус і показники виконання. Дії створення та редагування виконуються через форму з валідацією дат, що унеможливлює некоректні часові межі. Для кожного спринту доступні операції запуску, закриття та перенесення незавершених задач до backlog. Візуальна подача орієнтована на швидкий контроль стану ітерації та прозоре планування наступного циклу.

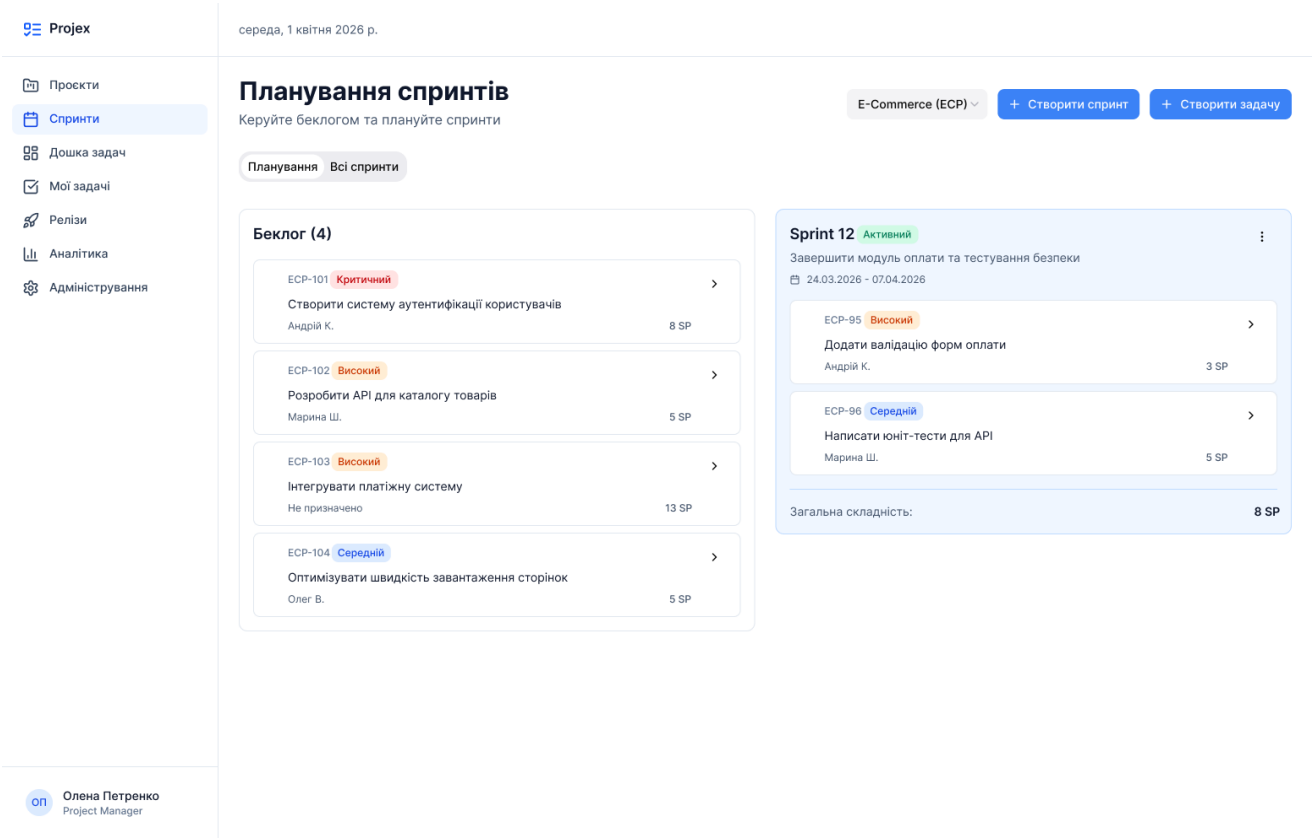

Рис. 3.5. Сторінка спринтів - планування

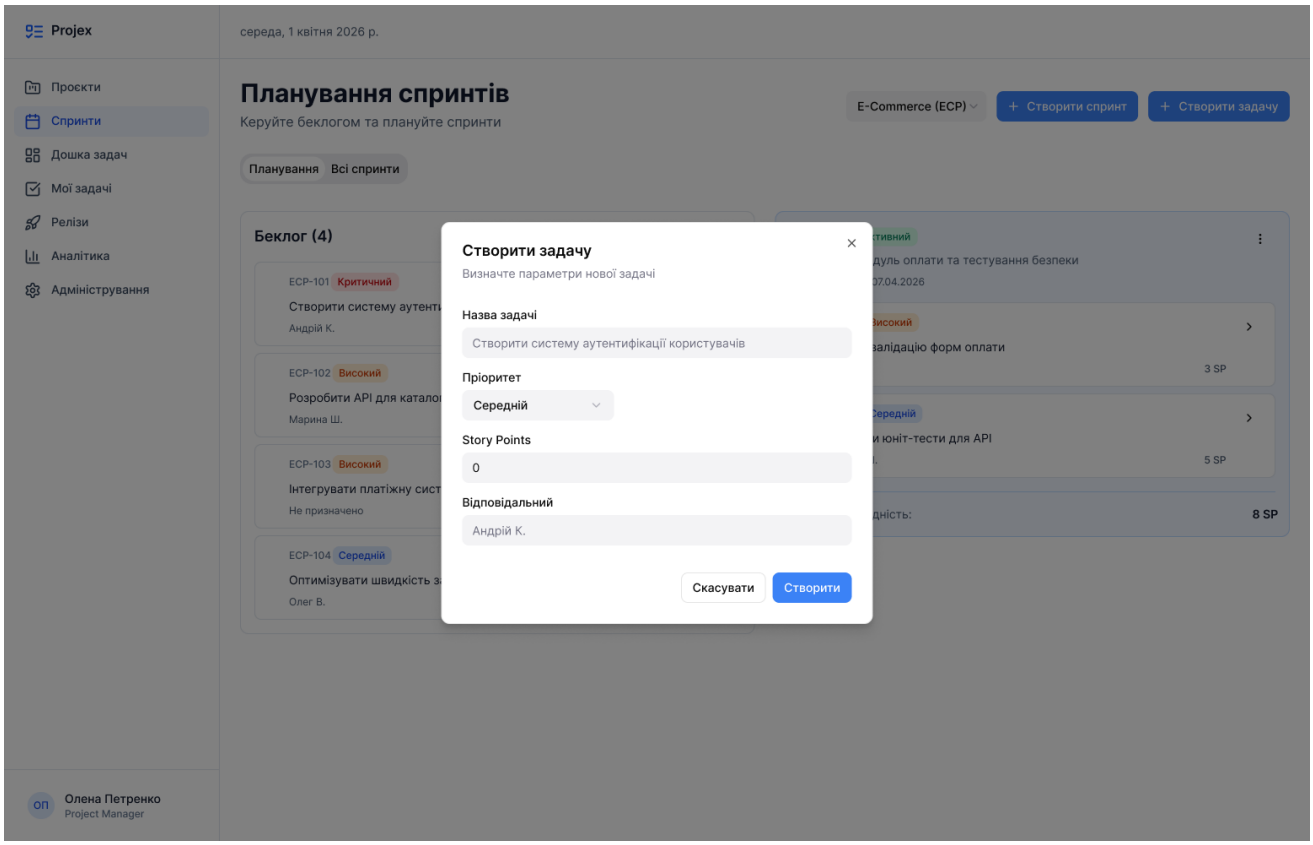

Рис. 3.6. Модальне вікно створення задачі

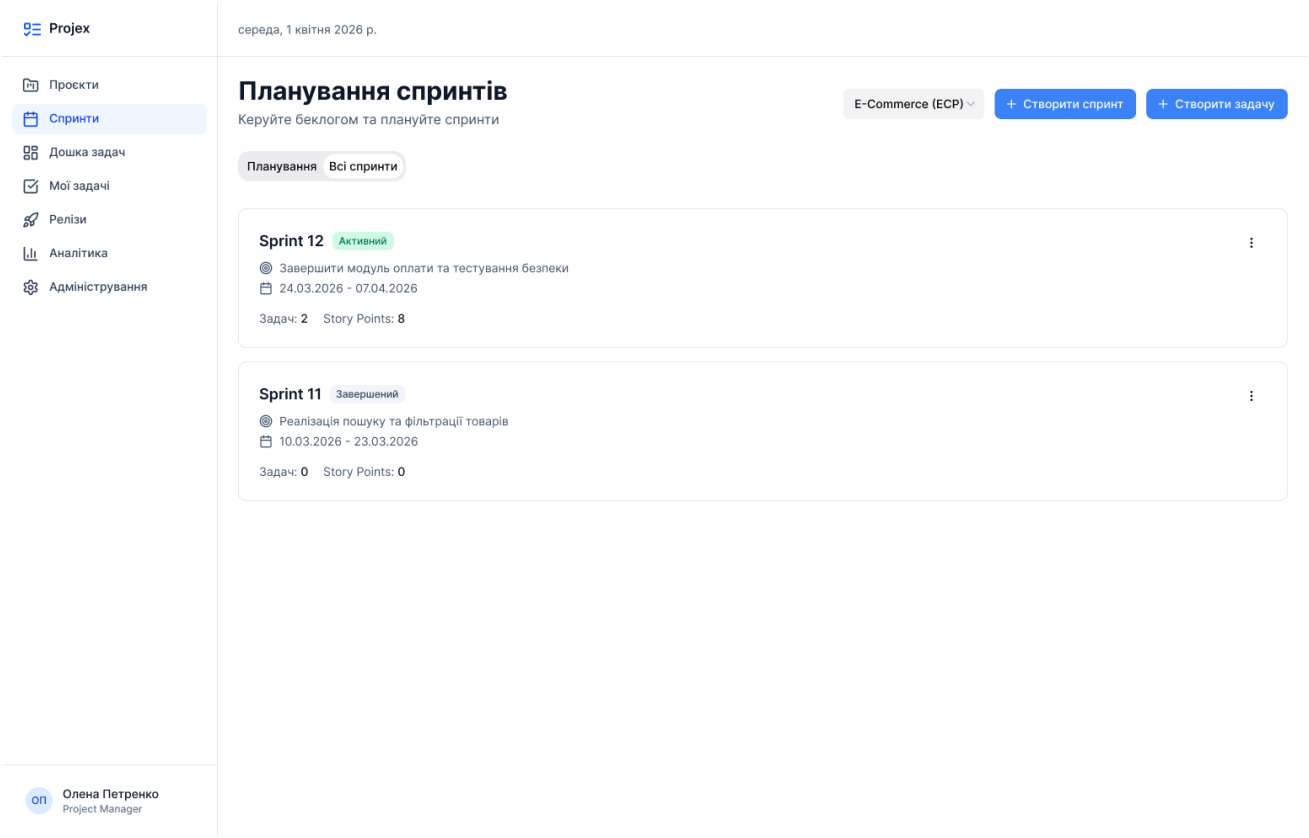

Рис. 3.7. Сторінка спринтів – всі спринти

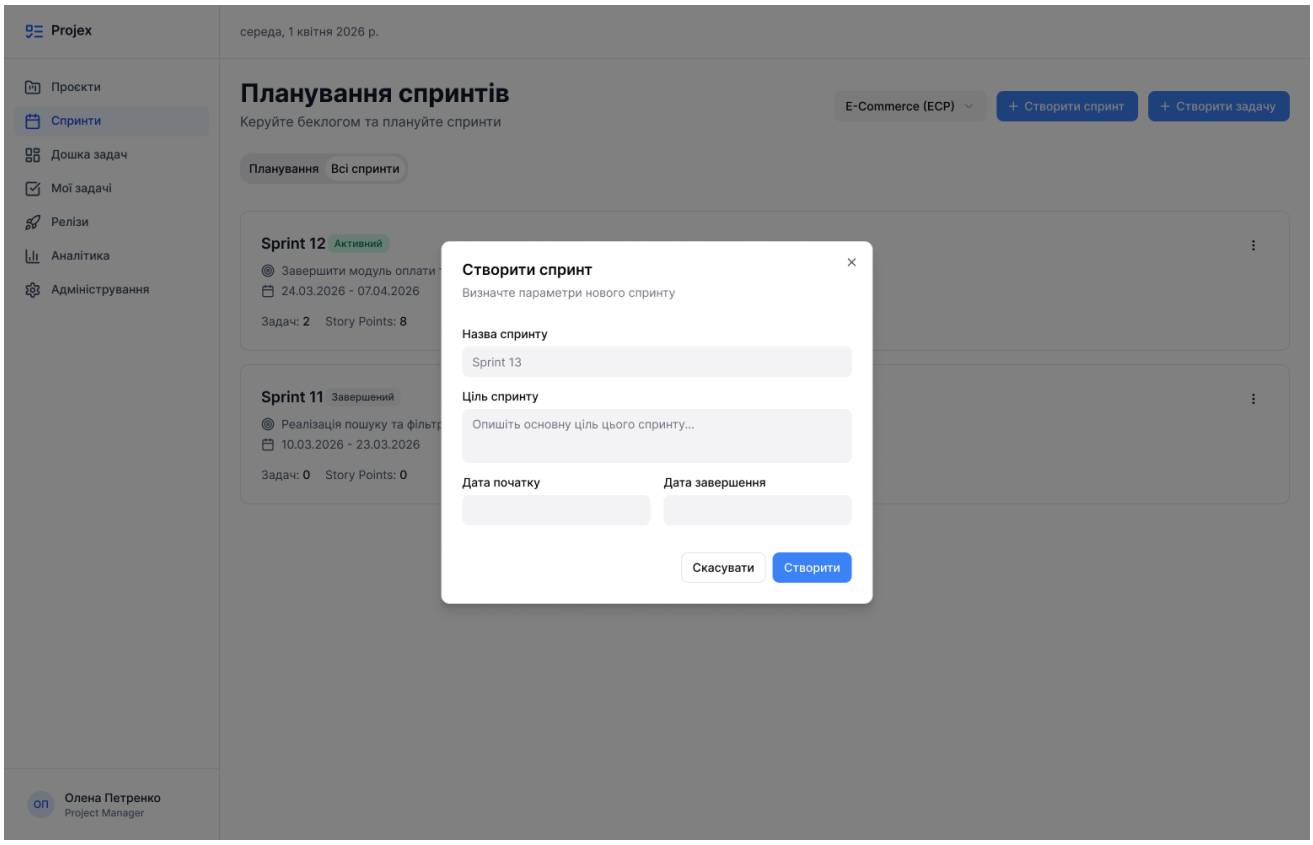

Рис. 3.8. Модальне вікно створення спринту

5. **Сторінка «Дошка задач»**

Kanban-інтерфейс побудовано на колонках життєвого циклу задач: To Do, In Progress, In Review, Done. Картка задачі містить коротку назву, пріоритет, виконавця, дедлайн і допоміжні маркери стану. Над дошкою розміщено панель фільтрів за проєктом, спринтом, виконавцем, пріоритетом і статусом, що забезпечує швидке фокусування на потрібному зрізі даних. Зміна статусу задачі відбувається інтерактивно, а UI оновлюється без перезавантаження сторінки.

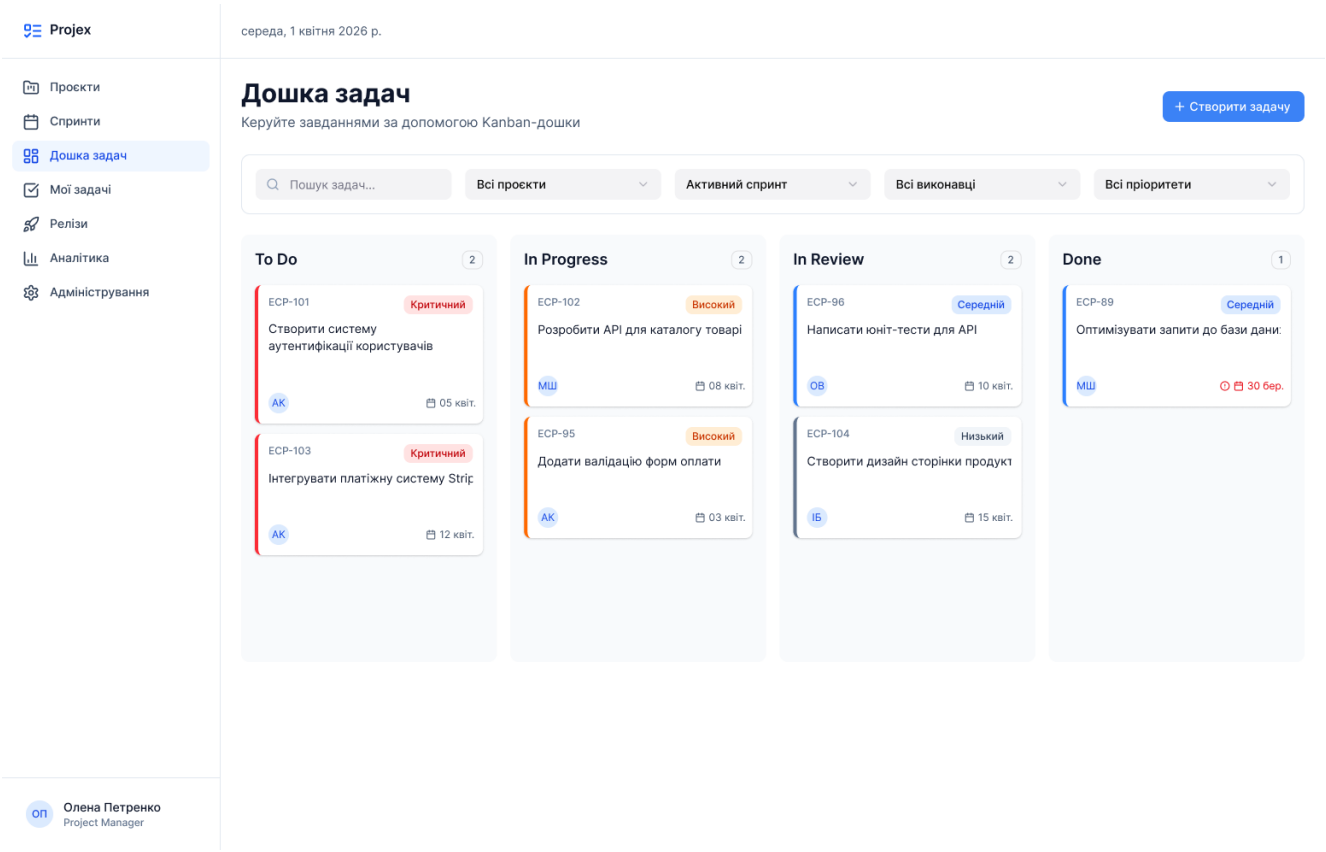

Рис. 3.9. Сторінка “Дошка задач”

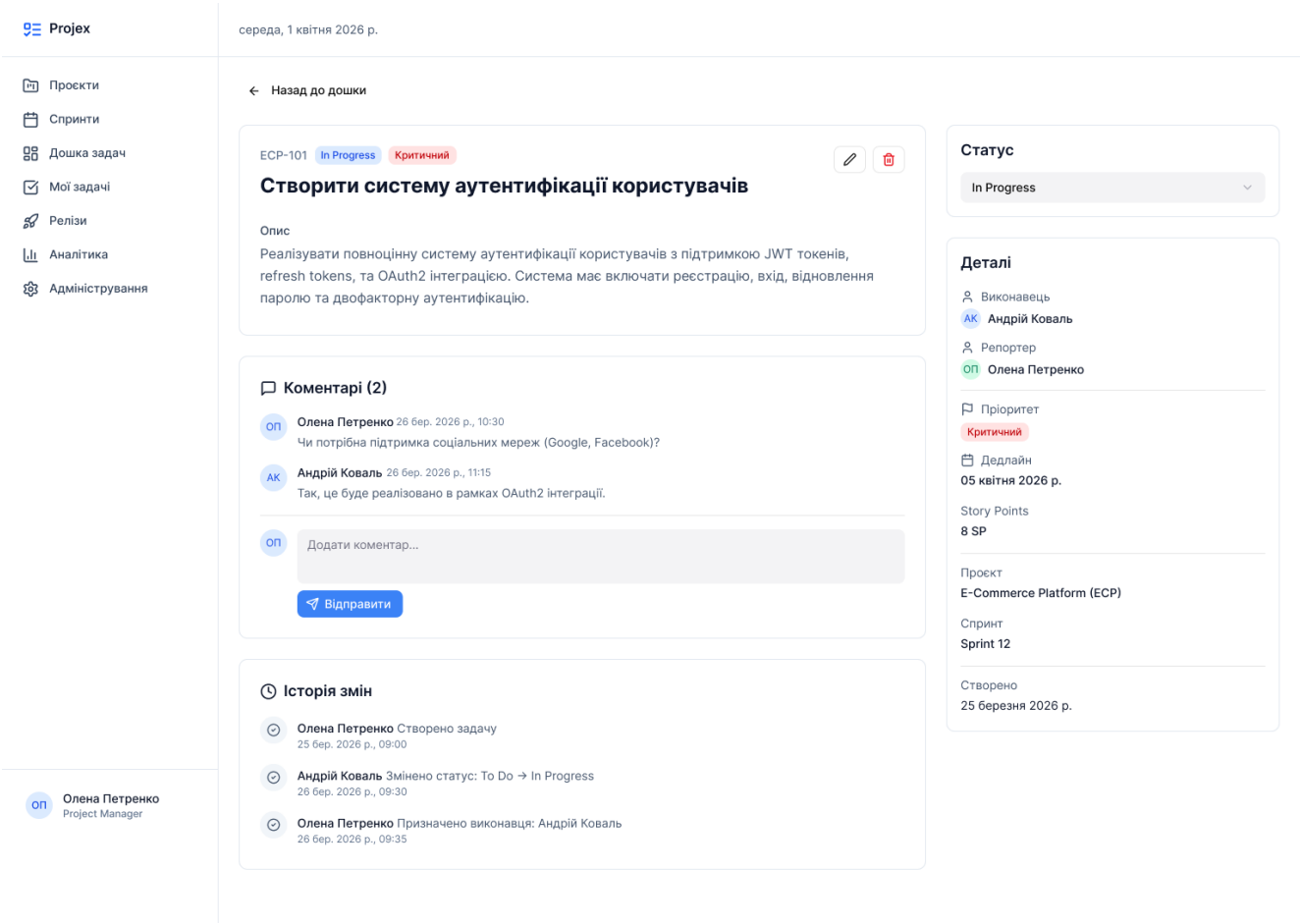

Рис. 3.10. Сторінка завдання

6. Сторінка «Мої задачі» (персональний робочий список)

Розділ надає персоналізований список задач поточного користувача у табличному або картковому форматі. Передбачено швидкі фільтри за статусом, пріоритетом та терміном виконання, а також виділення прострочених елементів. Зі сторінки доступні ключові дії: зміна статусу, перехід у деталі задачі, оновлення пріоритету. Такий формат дозволяє виконавцю концентруватися на власному навантаженні та поточних дедлайнах.

Projex

середа, 1 квітня 2026 р.

Проєкти
Спринти
Дошка задач
Мої задачі
Релізи
Аналітика
Адміністрування

Мої задачі

Всі задачі, призначені вам

To Do	In Progress	Прострочені	Завершені
2	2	0	1

Всі статуси | Всі пріоритети

Всі задачі (5) | Прострочені (0) | За пріоритетом

Ключ	Назва	Проєкт	Пріоритет	Статус	Дедлайн	SP
ECP-101	Створити систему аутентифікації користувачів	ECP	Критичний	In Progress	05 квіт.	8
ECP-95	Додати валідацію форм оплати	ECP	Високий	In Progress	03 квіт.	3
ECP-103	Інтегрувати платіжну систему Stripe	ECP	Критичний	To Do	12 квіт.	13
MBA-45	Рефакторинг компонентів UI	MBA	Середній	Done	30 бер.	5
ECP-110	Написати документацію для API	ECP	Низький	To Do	15 квіт.	2

ОП Олена Петренко
Project Manager

Рис. 3.11. Сторінка “Мої задачі”

Projex

середа, 1 квітня 2026 р.

Проєкти
Спринти
Дошка задач
Мої задачі
Релізи
Аналітика
Адміністрування

Мої задачі

Всі задачі, призначені вам

To Do	In Progress	Прострочені	Завершені
2	2	0	1

Всі статуси | Всі пріоритети

Всі задачі (5) | Прострочені (0) | За пріоритетом

Критичний (2)

Ключ	Назва	Проєкт	Пріоритет	Статус	Дедлайн	SP
ECP-101	Створити систему аутентифікації користувачів	ECP	Критичний	In Progress	05 квіт.	8
ECP-103	Інтегрувати платіжну систему Stripe	ECP	Критичний	To Do	12 квіт.	13

Високий (1)

Ключ	Назва	Проєкт	Пріоритет	Статус	Дедлайн	SP
ECP-95	Додати валідацію форм оплати	ECP	Високий	In Progress	03 квіт.	3

Середній (1)

Ключ	Назва	Проєкт	Пріоритет	Статус	Дедлайн	SP
MBA-45	Рефакторинг компонентів UI	MBA	Середній	Done	30 бер.	5

Низький (1)

Ключ	Назва	Проєкт	Пріоритет	Статус	Дедлайн	SP
ECP-110	Написати документацію для API	ECP	Низький	To Do	15 квіт.	2

ОП Олена Петренко
Project Manager

Рис. 3.12. Сторінка “Мої задачі” – фільтр по пріоритету

7. Сторінка «Релізи» (керування версіями)

Інтерфейс релізів відображає список сформованих версій із датою, пов'язаним спринтом і стислим описом змін. Форма створення релізу містить поля версії, дати публікації та release notes, а також перевірки на дублювання версій. Візуально блок релізів подано як структурований журнал змін, зручний для перегляду історії поставок. Це забезпечує прозорий зв'язок між виконаними задачами та підсумковими результатами ітерацій.

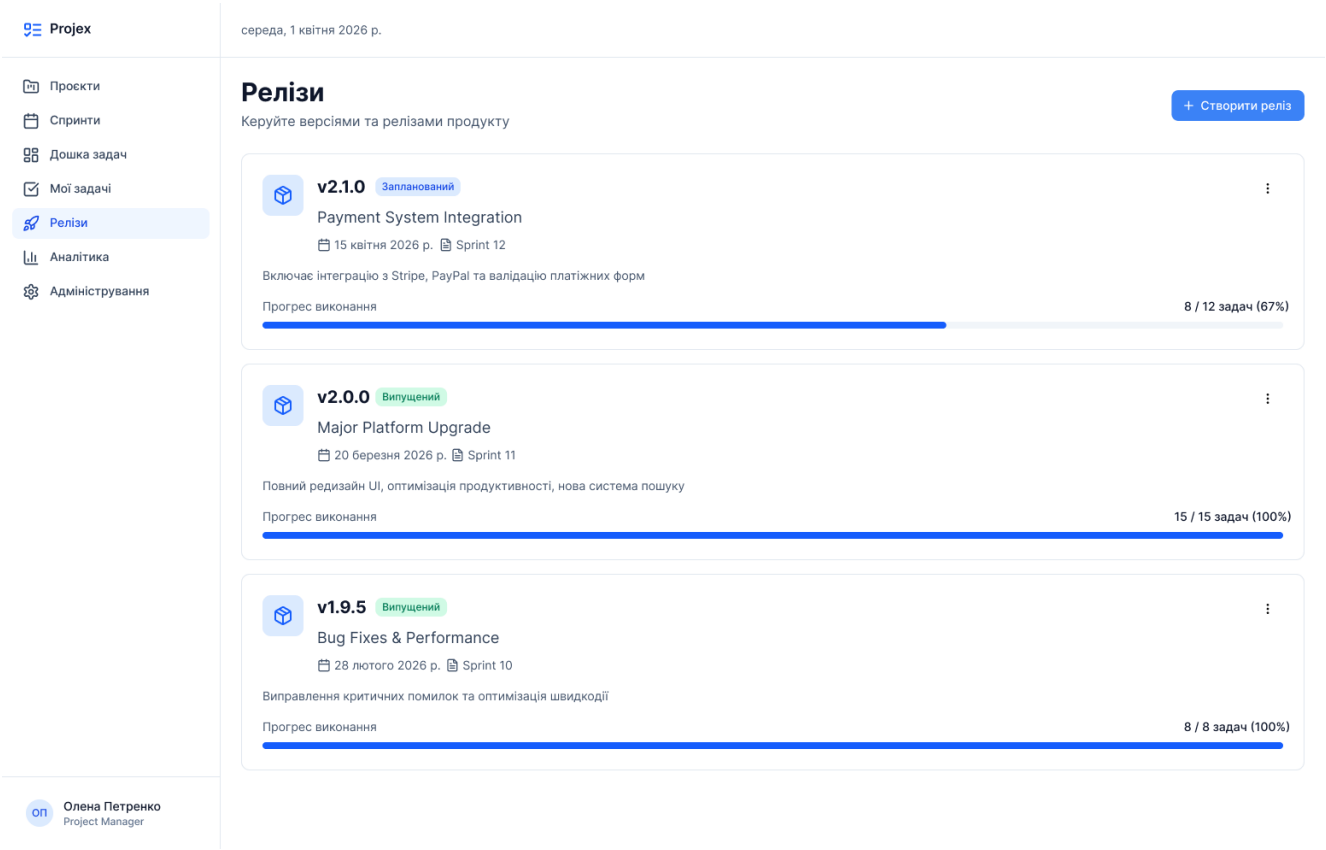

Рис. 3.13. Сторінка "Релізи"

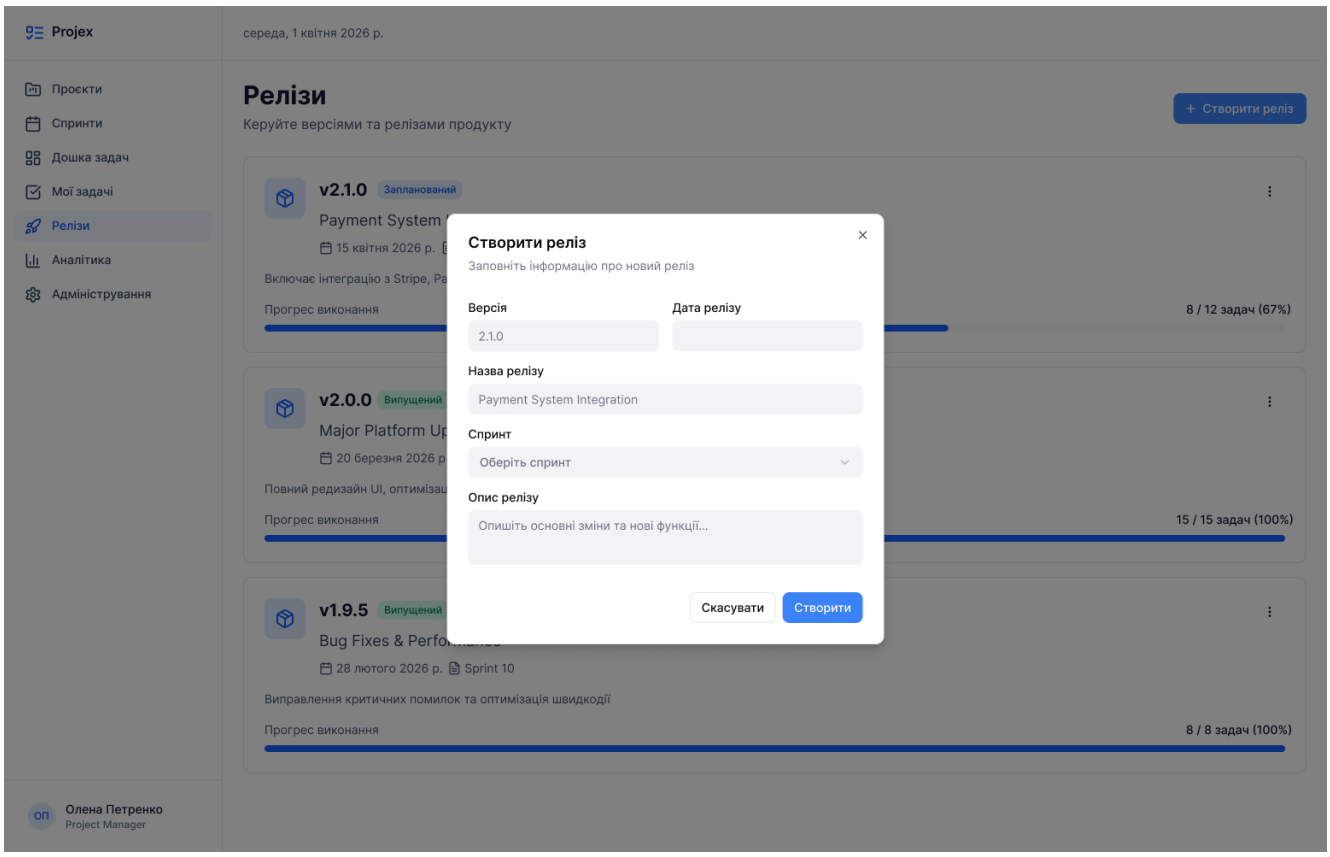

Рис. 3.14. Модальне вікно створення релізу

8. Сторінка «Аналітика» (дашборди і звіти)

Аналітичний розділ реалізовано у вигляді набору KPI-карток і графічних блоків із ключовими метриками: velocity, відсоток виконання, кількість прострочених задач, завантаження виконавців. Користувач може перемикати часові періоди й контексти (проєкт/спринт), отримуючи порівняльні зрізи. Для управлінських сценаріїв передбачено формування зведених звітів і експорт у PDF. Інтерфейс орієнтований на швидке прийняття рішень на основі фактичних даних.

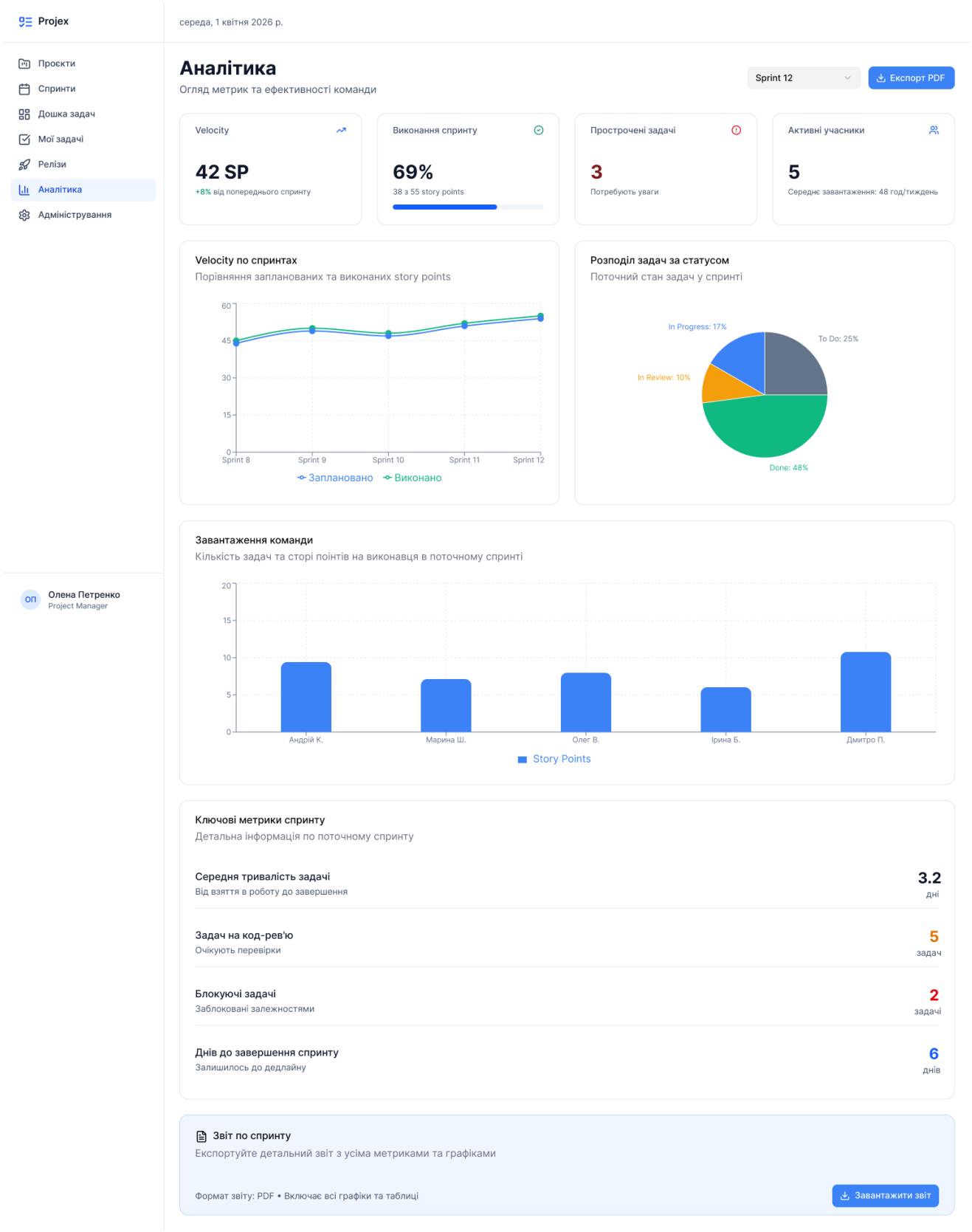

Рис. 3.15. Сторінка “Аналітика”

9. Сторінка «Профіль користувача» (персональні дані і активність)

Сторінка профілю містить картку користувача (аватар, ПІБ, роль, посада, дата приєднання) та блок статистики. У вкладці профілю реалізовано перегляд і редагування основних даних; роль відображається як інформаційне поле з поясненням щодо зміни прав через адміністратора. Блок статистики показує персональні агрегати (створені задачі, завершені задачі, проєкти, участь у командах). Така структура поєднує персоналізацію інтерфейсу з практичною робочою аналітикою.

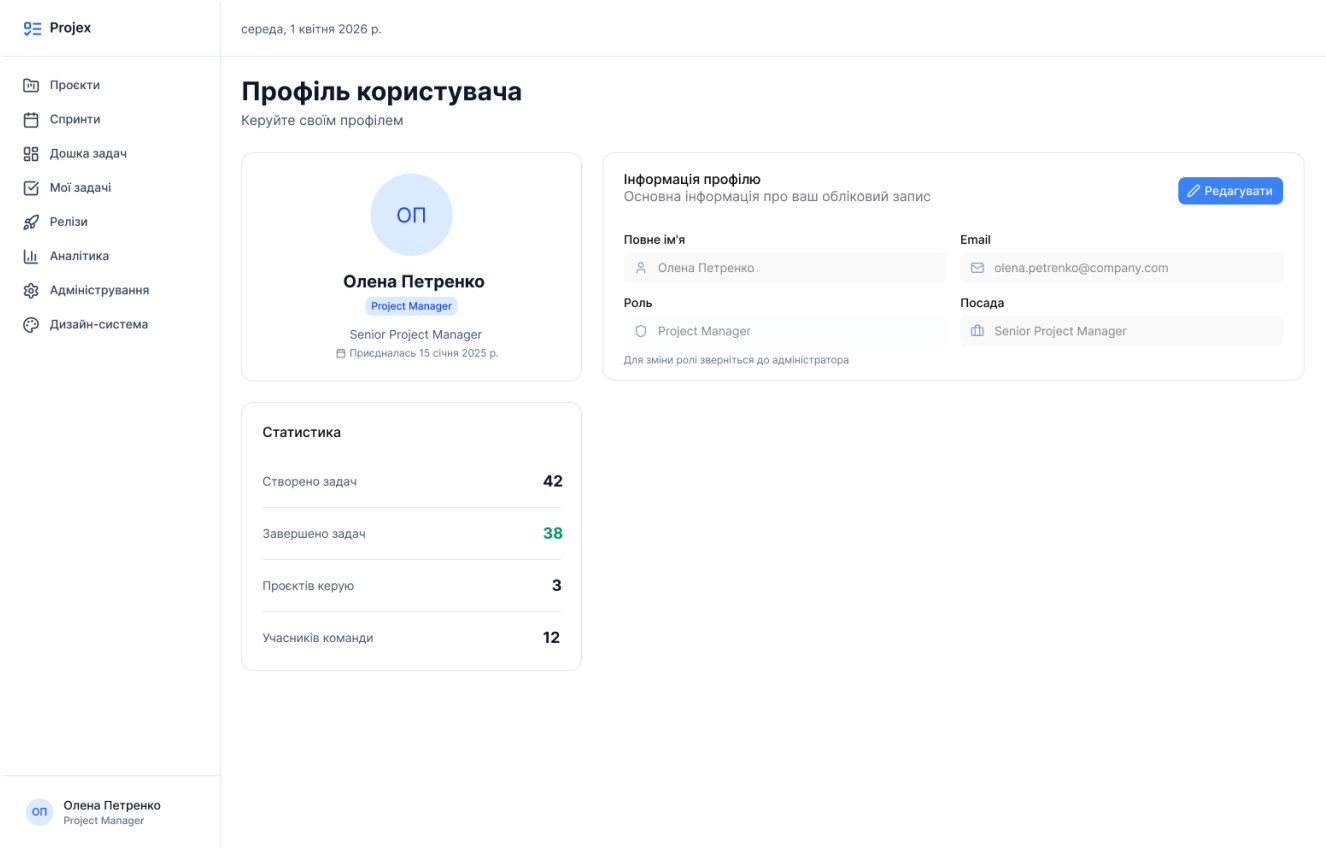

Рис. 3.16. Сторінка “Профіль користувача”

10. Сторінка «Адміністрування» (користувачі та ролі)

Адміністративний інтерфейс орієнтований на керування обліковими записами, ролями та базовими довідниками системи. Основним елементом є таблиця з пошуком, фільтрацією та контекстними діями для кожного запису. Форми створення/редагування передбачають перевірку обов’язкових полів і контроль унікальності ключових атрибутів. Розділ забезпечує централізоване керування доступом і підтримку цілісної ролеваї моделі.

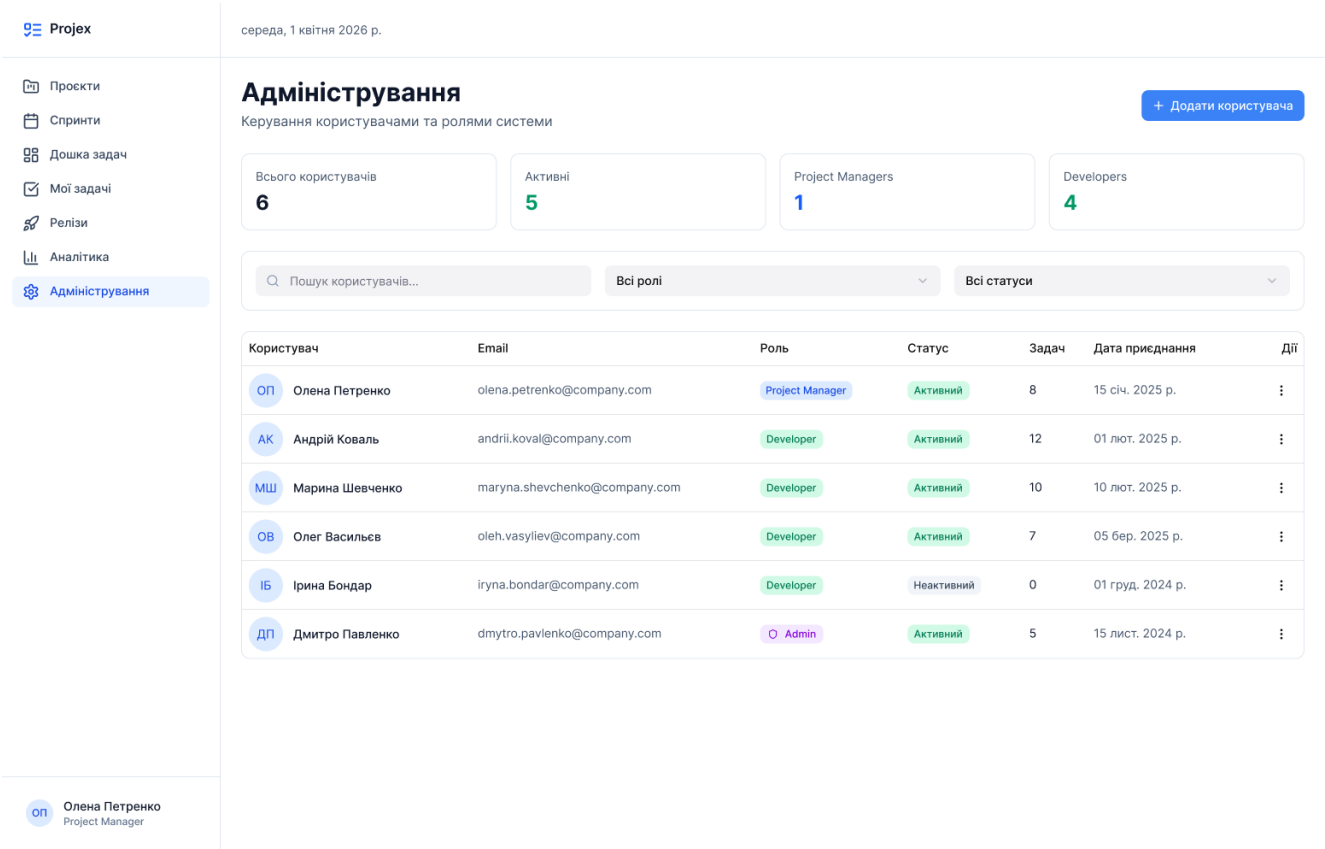

Рис. 3.17. Сторінка “Адміністрування”

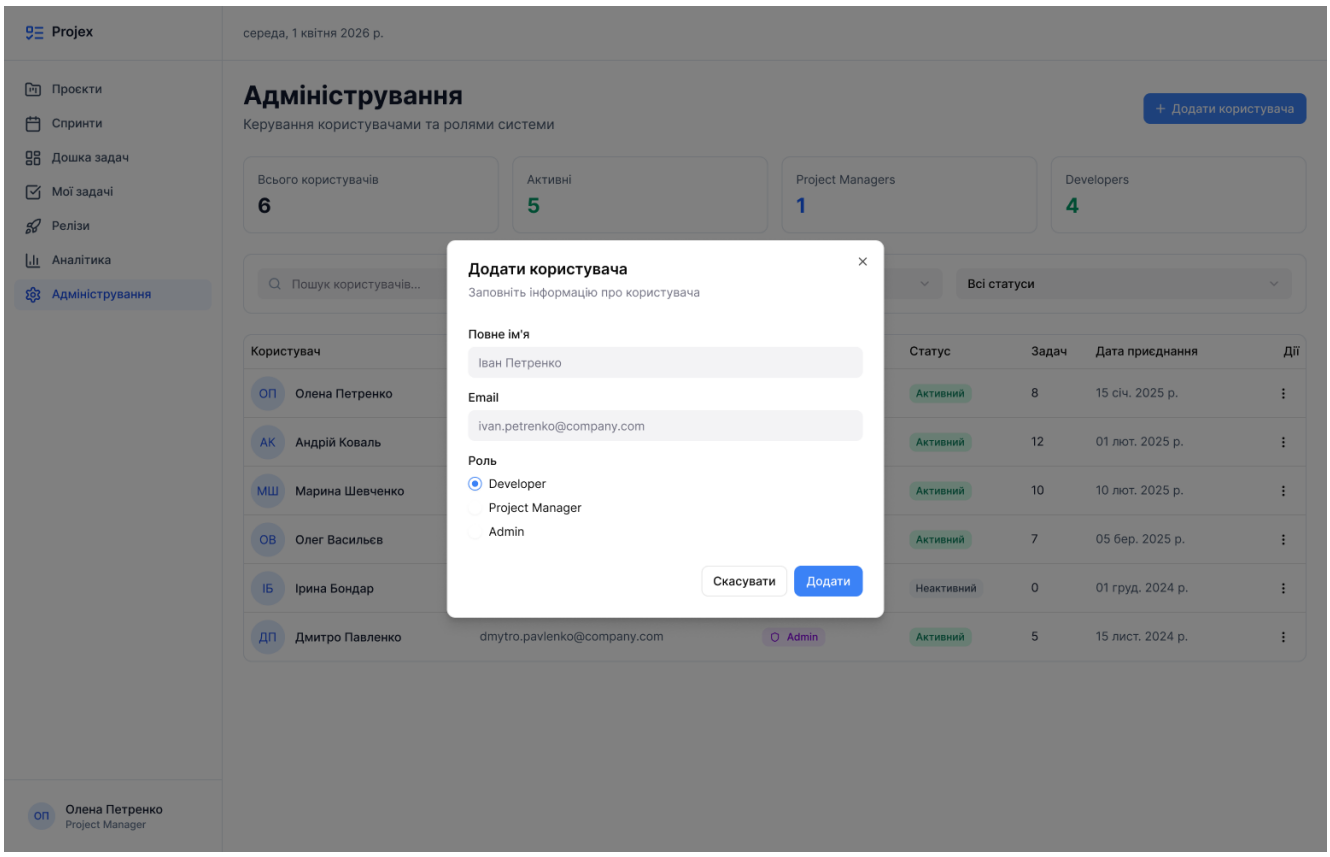

Рис. 3.18. Модальне вікно додавання користувача